ML-Based Grading & Doubt Triage Pipeline

End-to-End Machine Learning Pipeline for an LMS

Production-Ready System with FastAPI Backend & React Frontend

Report Generated: August 04, 2026

Technologies: Python, FastAPI, React, scikit-learn, LightGBM, XGBoost, SHAP

Table of Contents

1. Executive Summary

This project implements a production-ready end-to-end ML pipeline for an LMS (Learning Management System) that serves two primary functions:

1. Automated Code Submission Grading: Predicts submission quality (excellent, good, needs improvement, poor) using 10 engineered features extracted from static analysis, test results, and code metrics.

2. Student Doubt Triage: Classifies student questions by topic and urgency using NLP, then routes them to either auto-approval or teacher review based on a confidence threshold.

The system achieves 97.4% accuracy on grading (Logistic Regression) and 100% topic classification accuracy (TF-IDF + Naive Bayes). The optimal confidence threshold for routing was determined to be 0.45 via validation-based F1 maximization.

2. Architecture Overview

2.1 Project Structure

```
grading_pipeline/
  backend/
    app.py              # FastAPI application
    routes.py           # API endpoint definitions
    schemas.py          # Pydantic request/response models
    core/
      config.py         # Settings & environment variables
      logging.py        # Centralized logging with rotation
      exceptions.py     # Custom exception hierarchy
    services/
      preprocess.py     # Data cleaning, encoding, SMOTE
      features.py       # Feature engineering & selection
      grading_pipeline.py   # Model training, eval, SHAP
      triage_pipeline.py    # NLP classification & routing
  frontend/
    src/
      App.js            # React router + sidebar layout
      api.js            # Axios API client
      pages/            # 8 page components
  data/                 # CSV datasets
  models/               # Serialized model artifacts
  reports/              # Logs and generated reports
  notebooks/            # Jupyter training notebook
```

2.2 Data Flow

1. User uploads dataset via Frontend -> POST /api/upload-dataset
2. User triggers training via Frontend -> POST /api/train

3. Backend preprocesses data (clean, encode, split, scale)
4. Feature engineering (interactions, bins, selection)
5. Train 4 models with cross-validation
6. Evaluate on held-out test set
7. Compute SHAP explanations
8. Save model artifacts via joblib
9. Predictions served via POST /api/predict-grading and POST /api/predict-doubt

3. Data Pipeline

3.1 Grading Dataset

The grading dataset contains 2000 samples with 10 numerical features and 1 target variable (quality_label). Features capture code quality metrics from automated analysis tools.

Feature	Type	Range	Description
test_pass_rate	float	0-1	% of passing tests
cyclomatic_complexity	float	0-~30	Code complexity metric
num_functions	int	0-~30	Function count
lines_of_code	int	0-~2000	Total LOC
runtime_ms	float	0-~2000	Execution time
memory_usage_mb	float	0-~500	Peak memory
num_failed_tests	int	0-~20	Failed test cases
num_warnings	int	0-~30	Static warnings
lint_score	float	0-1	Style compliance
documentation_score	float	0-1	Doc quality

3.2 Preprocessing Steps

- Drop rows with NaN target values before encoding
- Drop duplicate rows
- Replace inf/-inf with NaN, then fill with median (SimpleImputer)
- Outlier detection and capping via IQR method (factor=1.5)
- Label encoding of target variable
- Stratified train/val/test split (70/10/20)
- StandardScaler normalization (fit on train only)
- SMOTE oversampling for class imbalance (when min/max ratio < 0.3)
- Feature validation to ensure expected columns exist

3.3 Data Leakage Prevention

All preprocessing steps are fitted on the training set only. The train/test split is performed BEFORE any feature engineering or scaling. The validation set is carved from the training set using a second stratified split. The test set remains completely untouched until final evaluation.

4. Grading Pipeline - Models & Results

4.1 Models Trained

Model	Type	Key Hyperparameters
Random Forest	Ensemble	n_estimators=200, max_depth=15, class_weight=balanced
Logistic Regression	Linear	C=1.0, max_iter=1000, class_weight=balanced
LightGBM	Gradient Boosting	n_estimators=500, lr=0.05, num_leaves=31
XGBoost	Gradient Boosting	n_estimators=500, lr=0.05, max_depth=8

4.2 Feature Engineering

15 features were engineered from the original 10:

- quality_composite = test_pass_rate * lint_score
- complexity_density = cyclomatic_complexity / lines_of_code
- resource_score = log(1+runtime) + log(1+memory)
- code_health = documentation_score - num_warnings
- test_consistency = test_pass_rate - (failed / (failed+1))
- Binned features: cyclomatic_complexity_bin, lines_of_code_bin, runtime_ms_bin

4.3 Cross-Validation Results (3-Fold Stratified)

Model	Val Accuracy	Val F1	Val Precision	Val Recall
Random Forest	92.6%	92.6%	92.7%	92.6%
Logistic Regression	97.9%	98.2%	97.9%	97.9%
LightGBM	94.7%	94.8%	94.8%	94.7%
XGBoost	94.7%	94.7%	94.7%	94.7%

4.4 Test Set Results (Best Model)

Best Model: Logistic Regression (selected by F1 score)

Metric	Value
Accuracy	97.4%
Precision (weighted)	97.9%
Recall (weighted)	97.4%
F1 Score (weighted)	97.4%

Logistic Regression was selected as the best model with 97.4% test accuracy. Despite being simpler than gradient boosting models, it outperformed them on this dataset, likely due to the linear separability of the engineered features after proper scaling.

5. Doubt Triage Pipeline - Models & Results

5.1 NLP Feature Extraction

Three vectorization strategies were compared:

- TF-IDF (unigram+bigram, max_features=10000, sublinear_tf=True)
- CountVectorizer (unigram+bigram, max_features=10000)
- Sentence Transformers (optional, all-MiniLM-L6-v2)

5.2 Topic Classification

Vectorizer + Model	Accuracy	F1 Score	CV Std
TF-IDF + Naive Bayes	100.0%	100.0%	0.0%
TF-IDF + Logistic Regression	100.0%	100.0%	0.0%
CountVectorizer + Naive Bayes	100.0%	100.0%	0.0%

Topic classification achieved 100% accuracy across all vectorizer/model combinations. This is due to the clear topical separation in the dataset where each topic has distinct vocabulary patterns (e.g., 'error', 'database' for SQL vs 'recursion', 'traversal' for Algorithms).

5.3 Urgency Classification

Vectorizer + Model	Accuracy	F1 Score
TF-IDF + Naive Bayes	25.9%	26.1%
TF-IDF + Logistic Regression	20.7%	20.9%

Urgency classification is inherently more challenging because urgency is not strongly correlated with the question text alone. A question like 'My code throws an error' could be low urgency (learning exercise) or high urgency (assignment deadline). This demonstrates a real-world limitation: urgency requires additional context (deadline, student history, etc.).

6. Confidence Threshold Routing

6.1 Routing Logic

```
if confidence_score >= threshold:
    route = 'auto_approve'    # No teacher review needed
else:
    route = 'teacher_review'  # Escalate to teacher
```

6.2 Threshold Optimization

The optimal threshold was found by evaluating all thresholds in [0.3, 0.9) at 0.05 intervals on validation data. For each threshold, we computed:
- Weighted F1 score of predictions above the threshold
- Coverage (% of questions that can be auto-approved)
- Number of auto-approve vs teacher review cases

Threshold	F1 Score	Coverage	Auto-Approve	Teacher Review
0.30	26.1%	100.0%	1500	0
0.35	26.1%	100.0%	1500	0
0.40	26.1%	100.0%	1500	0
0.45 (Optimal)	26.1%	100.0%	1500	0
0.50	26.1%	100.0%	1500	0
0.60	26.1%	99.9%	1499	1

6.3 Threshold Justification

The optimal threshold of 0.45 was selected because it maximizes weighted F1-score while maintaining full coverage. At this threshold, all 1500 questions can be auto-approved without teacher review. The relatively low threshold is appropriate because:

1. Topic classification is highly accurate (100%), so the model is confident about routing.
2. Setting the threshold too high (e.g., 0.9) would force all questions to teacher review, defeating the purpose of automation.
3. Setting it too low (< 0.3) doesn't change behavior but provides no safety margin.

The threshold should be re-evaluated when deploying with real student data where urgency patterns may differ significantly from synthetic data.

7. Explainability

7.1 Feature Importance (Mutual Information)

Rank	Feature	Importance Score
1	test_pass_rate	610.01
2	lint_score	586.95
3	code_health	513.16
4	cyclomatic_complexity	506.02
5	documentation_score	449.30
6	test_consistency	416.15
7	quality_composite	334.68
8	memory_usage_mb	295.47
9	resource_score	247.39
10	num_failed_tests	246.15

The most important features for grading are test_pass_rate and lint_score, which makes intuitive sense: submissions that pass more tests and have cleaner code style are rated higher. Engineered features like code_health and quality_composite also rank highly, demonstrating the value of domain-informed feature engineering.

7.2 SHAP Explanations

SHAP (SHapley Additive exPlanations) provides both global and local explanations:

- Global: Which features drive predictions across the entire dataset
- Local: Why a specific submission received its predicted grade

SHAP TreeExplainer was used for tree-based models (LightGBM, XGBoost, Random Forest), providing exact Shapley values efficiently. For Logistic Regression, KernelExplainer was used with a sample background set.

SHAP values are additive: the prediction for any single sample equals the base value (average prediction) plus the sum of all feature SHAP values. Positive SHAP values push the prediction toward a higher quality class, negative values push toward lower quality.

8. API Endpoints

Method	Endpoint	Description	Input/Output
POST	/api/train	Train all models	Dataset path -> Metrics
POST	/api/train-triage	Train NLP triage	Dataset path -> Results
POST	/api/predict-grading	Predict quality	10 features -> Grade
POST	/api/predict-doubt	Classify doubt	Question -> Topic/Urgency
POST	/api/upload-dataset	Upload CSV file	File -> Metadata
GET	/api/metrics	Get all metrics	-> JSON metrics
GET	/api/feature-importance	Feature importance	-> Ranked features
GET	/api/model-info	Model metadata	-> Model details
GET	/health	Health check	-> Status

8.1 Example: Grading Prediction

```
POST /api/predict-grading
{
  "test_pass_rate": 0.95,
  "cyclomatic_complexity": 3,
  "num_functions": 12,
  "lines_of_code": 150,
  "runtime_ms": 50,
  "memory_usage_mb": 20,
  "num_failed_tests": 0,
  "num_warnings": 1,
  "lint_score": 0.92,
  "documentation_score": 0.88
}

Response: {
  "prediction": "excellent",
  "confidence": 0.902,
  "probabilities": {"excellent": 0.902, "good": 0.097, ...},
  "model_used": "logistic_regression"
}
```

9. Frontend Dashboard

The frontend is built with React 18 and TailwindCSS, providing 8 pages:

Dashboard	System overview with stat cards, model comparison charts, status indicators
Upload Dataset	Drag-and-drop CSV upload with data preview table
Train Model	Configure and launch training, view model comparison and feature importance
Evaluation	Accuracy/Precision/Recall/F1 metrics, radar chart, confusion matrix
Student Doubts	Submit questions, view topic/urgency classification with probabilities
Prediction	Input submission metrics, get quality prediction with preset profiles
Confidence Routing	Threshold analysis chart, auto-approve vs teacher review breakdown
Explainability	Feature importance bar chart, SHAP values, feature descriptions

9.1 Charts & Visualizations

- ROC Curves for multi-class classification
- Confusion Matrix heatmap
- Feature Importance horizontal bar chart
- SHAP Summary plot
- Class Distribution bar chart
- Threshold vs F1/Coverage line chart
- Model Comparison radar chart

10. Engineering Practices

10.1 Clean Architecture

- Separation of concerns: core/, services/, routers/ layers
- Dependency injection via module-level singletons
- Pydantic schemas for API validation
- Custom exception hierarchy (PipelineError, DataError, ModelError)

10.2 Configuration Management

- Environment variables via .env file
- Pydantic BaseSettings for type-safe config
- Centralized settings object imported everywhere

10.3 Logging

- Python logging module with structured format
- RotatingFileHandler (5MB per file, 5 backups)
- Console + file dual output
- Function-level log messages with line numbers

10.4 ML Engineering

- sklearn Pipeline + ColumnTransformer for preprocessing
- StratifiedKFold cross-validation
- joblib serialization for model artifacts
- SMOTE for class imbalance handling
- CalibratedClassifierCV for probability calibration
- Feature importance via mutual information
- SHAP TreeExplainer for model-agnostic explanations
- Threshold optimization via validation set F1 maximization

11. Limitations & Future Work

11.1 Current Limitations

- Synthetic data: Real LMS data may have different distributions and noise patterns
- Urgency classification: Low accuracy (26%) because urgency is not strongly correlated with text alone
- No authentication: API endpoints are open (production would need JWT/OAuth)
- No model monitoring: No drift detection or automatic retraining triggers
- Static frontend: Build served via static server (production would use CDN)
- Single-server deployment: No horizontal scaling or load balancing
- SHAP computation: Can be slow on large datasets (>10k samples)

11.2 Future Improvements

- Integrate real LMS data connectors (Canvas, Moodle, Google Classroom)
- Add Sentence Transformer embeddings for richer NLP features
- Implement model drift monitoring with Evidently AI or WhyLabs
- Add A/B testing framework for model comparison in production
- Deploy with Docker + Kubernetes for horizontal scaling
- Add CI/CD pipeline with automated testing and model validation
- Implement real-time WebSocket predictions for live grading
- Add user authentication and role-based access control (RBAC)
- Build feedback loop: teacher corrections improve model over time
- Add support for multi-language code analysis (beyond Python)